



AdaptiveRoute

Operational route intelligence

Agentic replanning

A fine-tuned routing policy for speed.
A MILP solver for the guarantee.

PROOF OF CONCEPT · 5 DAYS

Gabriel Carvalho Domingos da Conceição

Routes are planned once. Reality changes all day.

A delivery route is optimised the night before. By mid-morning it is already wrong.

NETWORK

A road closes

An accident blocks the segment the driver was routed through.

DEMAND

A customer cannot receive

Store shut, nobody on site, delivery window missed.

PRIORITY

The order changes

A delivery becomes urgent after the plan was frozen.

Today the driver phones an operator, who re-plans by hand. **The optimisation asset built overnight is abandoned by 10am.**

Two obvious approaches, each with a disqualifying flaw

OPTION A

Ask a language model

Speaks the driver's language. Answers in seconds.

But: no notion of capacity. It will route straight through the blocked road.

OPTION B

Re-run the optimiser

Provably correct. Respects every constraint.

But: it needs a formal model of the disruption, not a sentence.

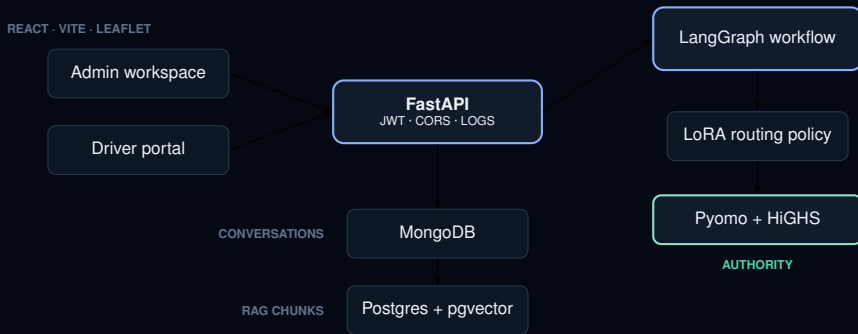
The gap is not optimisation, and not language. **It is the handoff.**

The model proposes. The solver decides.

The fine-tuned model is treated as a **fast candidate generator**, never as an authority. Every plan it produces is parsed, checked against deterministic rules, repaired if it can be repaired safely, and replaced by the MILP solver if it cannot.

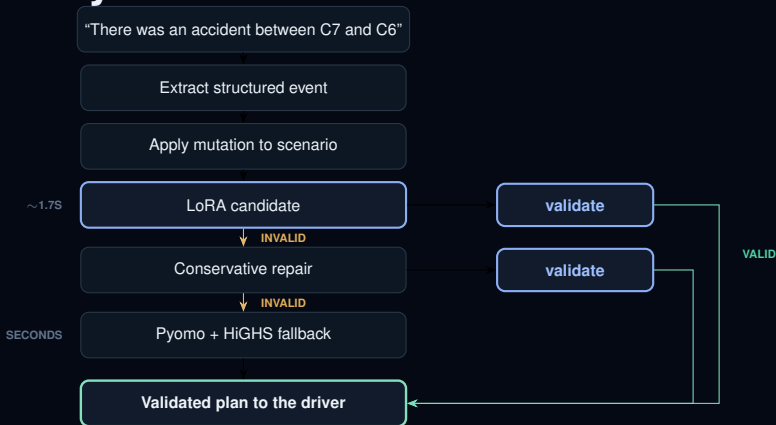
Nothing reaches the driver without passing the same validation gate — **regardless of who produced it.**

FastAPI core, pluggable everywhere it matters



Engine, policy and event extractor are swappable by environment variable. Tests run in-memory — no GPU, no database, no model server.

Each stage is slower and more trustworthy than the one above it



When the existing plan survives the event, it is reused and the model is never called.

One arbiter, independent of who wrote the plan

A PLAN IS ACCEPTED ONLY IF EVERY RULE HOLDS

- Every active customer visited **exactly once**
- Each route starts and ends at the depot
- Vehicle capacity respected
- No blocked arc traversed
- Only known vehicles and active customers
- No duplicate or inactive stops

NUMBERS CANNOT BE HALLUCINATED

The validator **recomputes** load and distance.

FAILURE IS A PROMPT, NOT AN ERROR

Violations feed back. Three attempts, then the solver.

Supervised fine-tuning where the optimiser is the teacher

HOW TRAINING DATA IS MADE

1. Generate a synthetic CVRP instance
2. Solve it to optimality with HiGHS
3. Inject an operational event
4. **Re-solve** the disrupted scenario
5. Validate, then serialise the pair

The solver labels its own training set. The model learns to imitate optimal replanning, not to invent it.

CONFIGURATION

Base model	Qwen2.5-7B-Instruct
Method	LoRA / QLoRA
Rank r	16
Alpha	32
Dropout	0.05
Quantisation	4-bit NF4
Compute dtype	bfloat16
Epochs	1
Hardware	RTX 3090 · 24 GB

Four variants, one fixed benchmark

1,000 held-out scenarios, identical across every run.

	TRAINING DATA	FEASIBLE	CAPACITY	BLOCKED ARC	EXACT
v3	30K compact curriculum	91.7%	72	11	4.6%
v4	+ 70K hard continuation	92.9%	61	10	5.8%
v5	+ 20K error-driven	94.4%	46	9	5.6%
v6	+ 10K further error-driven	94.0%	50	10	5.4%

v5 selected. Highest feasibility, fewest violations of both kinds, 100% valid JSON everywhere. v6 confirmed the curve had flattened.

Where the model stops working

Measured, not estimated.

CUSTOMERS	VALID	FAILURE MODE
8	100%	—
9	100%	—
10	60%	customer omitted
12	33%	customer omitted
14	0%	customer omitted

Small samples: 3–5 scenarios per size.

THE BOUNDARY

8–9 customers

Above it the model omits customers — it does not route them badly.

A **cardinality limit**, not a routing-quality problem.
The fix is training data at 10–24 customers.

Beyond the boundary the plan is still correct —
the cascade falls through to the solver.

What was built to make the result trustworthy

73

Python modules,
~11,800 lines

24

test files, runnable
without GPU or database

11

architecture and
evaluation documents

- Immutable domain model throughout
- Service / repository separation
- Protocol interfaces at every seam
- CI split: fast API and forked solver tests
- bcrypt credentials, signed JWT, scoped CORS
- Structured logs with request ids

What we would fix before anyone depends on this

SCALE

MTZ relaxation is weak. Solver calls still run inside the request.

DATA

Fully synthetic. No real traffic, telemetry or road network.

THE UNGUARDED EDGE

Validation covers the plan, not the question

It proves the plan is right *for the mutated scenario* — not that the mutation matches what the driver said.

This is the next thing to close.

In priority order

1. Harden event extraction
2. Push the cardinality boundary
3. Move optimisation off the request path
4. Instrument the cascade in production
5. Validate against real operations

The system stays correct when the model is wrong

- **94.4% feasible replans** at the size it was trained for
- The remaining **5.6% is harmless** — validation, repair, fallback
- A measured boundary of **8–9 customers**, not an assumed one
- Natural language to validated route, **end to end**

Built in five days. The interesting result is not the model — it is everything built around it.